1. Operating System Overview

Computer and Virtual Technologies Engineering
Software Engineering

Operating Systems

2025-2026

Universidad LOYOLA

Contents

1) Objectives and functions
2) The evolution of operating systems
3) Major achievements
   - Processes
   - Memory management
   - Information protection and security
   - Scheduling and resource management
4) Features of modern operating systems
   - Microkernel architecture
   - Multithreading
   - Symmetric multiprocessing
   - Distributed operating systems
5) UNIX systems

Contents

1) Objectives and functions
2) The evolution of operating systems
3) Major achievements
   - Processes
   - Memory management
   - Information protection and security
   - Scheduling and resource management
4) Features of modern operating systems
   - Microkernel architecture
   - Multithreading
   - Symmetric multiprocessing
   - Distributed operating systems
5) UNIX systems

Operating system (OS) objectives and functions

- An OS is a **program (software)** that controls the execution of application programs, and acts as an interface between **applications** and **the computer hardware**.
- Objectives:

  1-Convenience.

  2- Efficiency.

  3- Ability to evolve.

Operating system (OS) objectives and functions

- An OS is a **program (software)** that controls the execution of application programs, and acts as an interface between **applications** and **the computer hardware**.
- Objectives:

  1-Convenience.

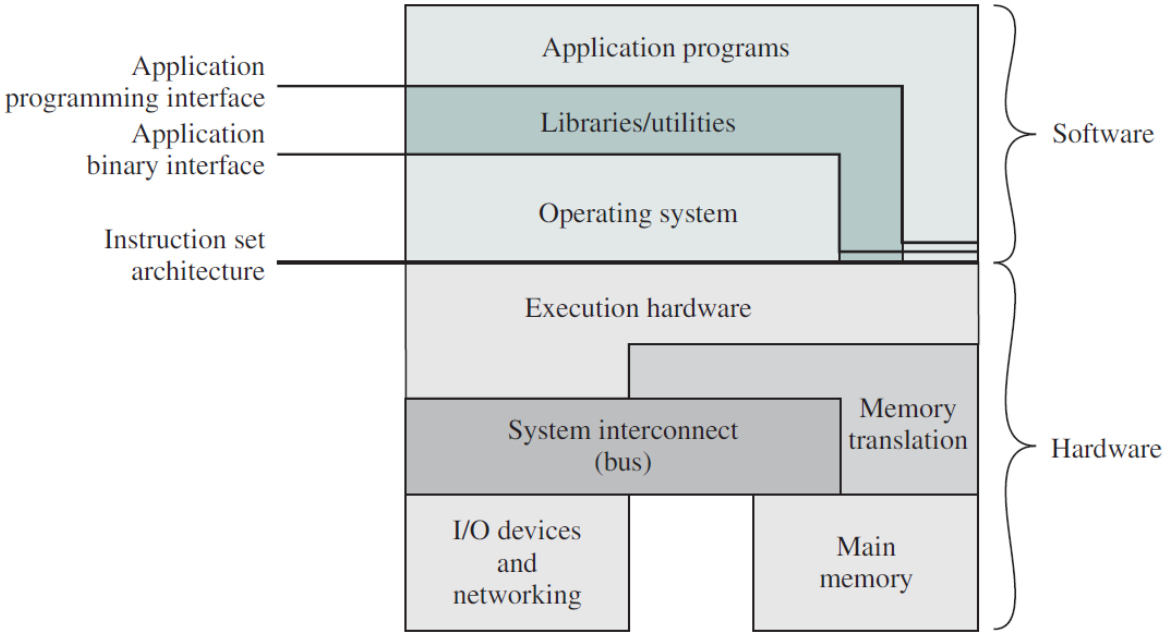
  2- Efficiency.

  3- Ability to evolve.

1- The operating system as a user/computer interface.

Computer hardware and software structure viewed in a layered fashion.

The OS masks the details of the hardware from the programmer and provides the programmer with a convenient interface for using the system.



Operating system (OS) objectives and functions

- An OS is a **program (software)** that controls the execution of application programs, and acts as an interface between **applications** and **the computer hardware**.
- Objectives:

  1-Convenience.

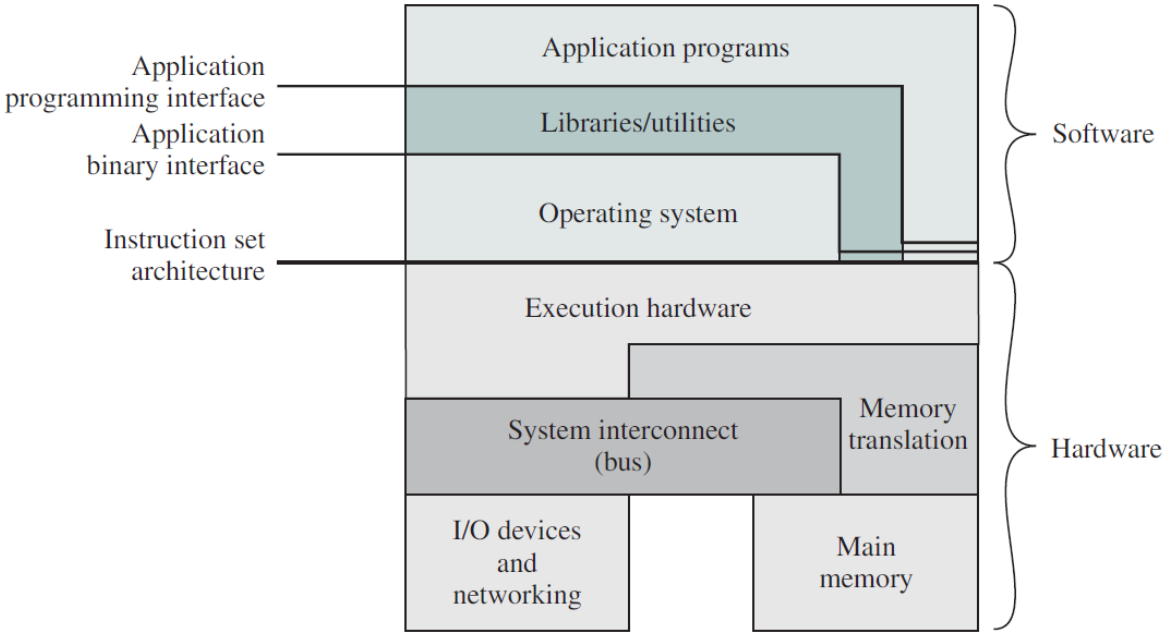
  2- Efficiency.

  3- Ability to evolve.

1- The operating system as a user/computer interface.

OS provides services in the following areas:

Program development, program execution, access to I/O devices, controlled access to files, system access, error detection and response, accounting, instruction set architecture (ISA), application binary interface (ABI) and application programming interface (API).



Operating system (OS) objectives and functions

- An OS is a **program (software)** that controls the execution of application programs, and acts as an interface between **applications** and **the computer hardware**.
- Objectives:

  1-Convenience.

  2- Efficiency.

  3- Ability to evolve.

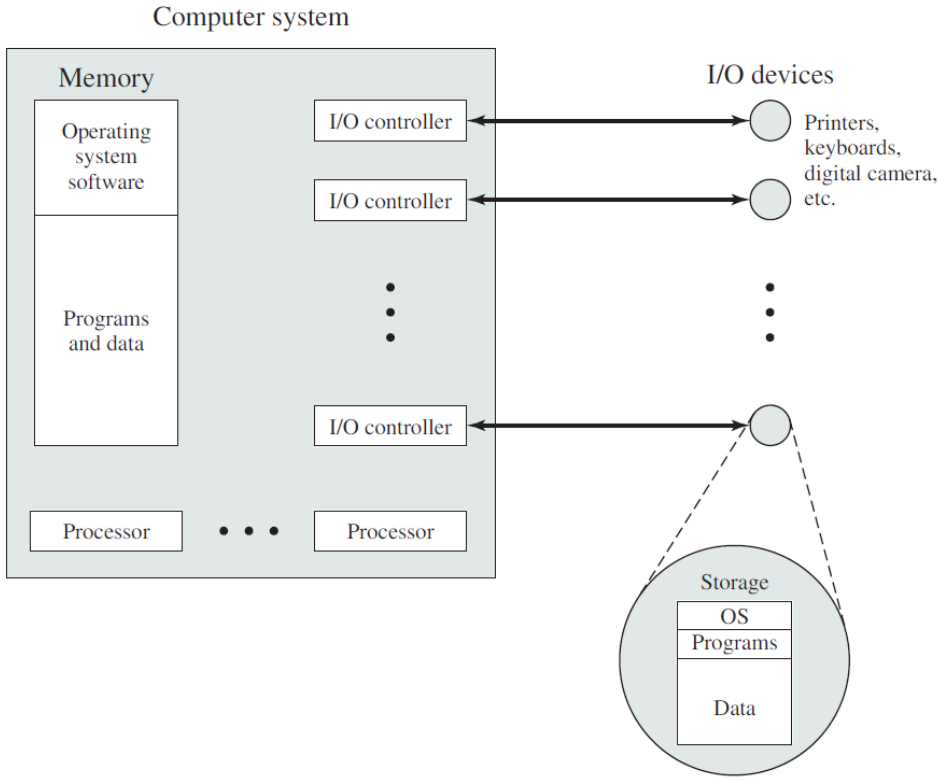
2- The operating system as resource manager.

The OS is responsible for controlling the use of a computer´s resources, such as **I/O**, **memory** and **processor execution** time.

Control mechanisms:

- The OS functions in the same way as ordinary computer software. It is a suite of programs executed by the processor.
- The OS frequently relinquishes control and depend on the processor to regain control.

Kernel or **nucleus** contains the most frequently used functions in the OS.



Operating system (OS) objectives and functions

- An OS is a **program (software)** that controls the execution of application programs, and acts as an interface between **applications** and **the computer hardware**.
- Objectives:

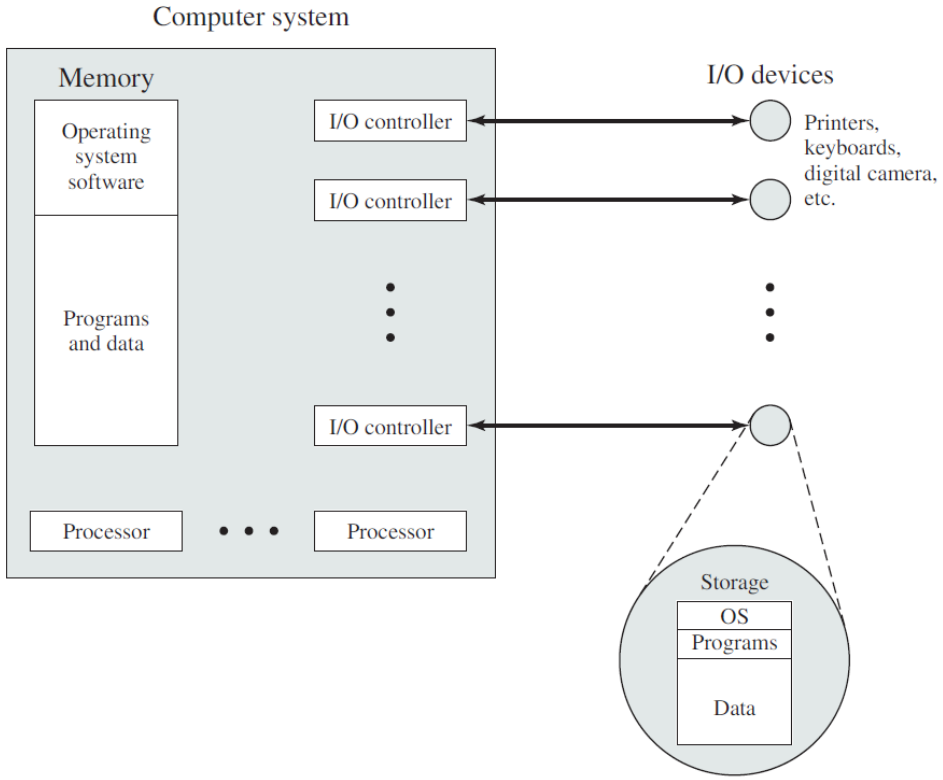
  1-Convenience.

  2- Efficiency.

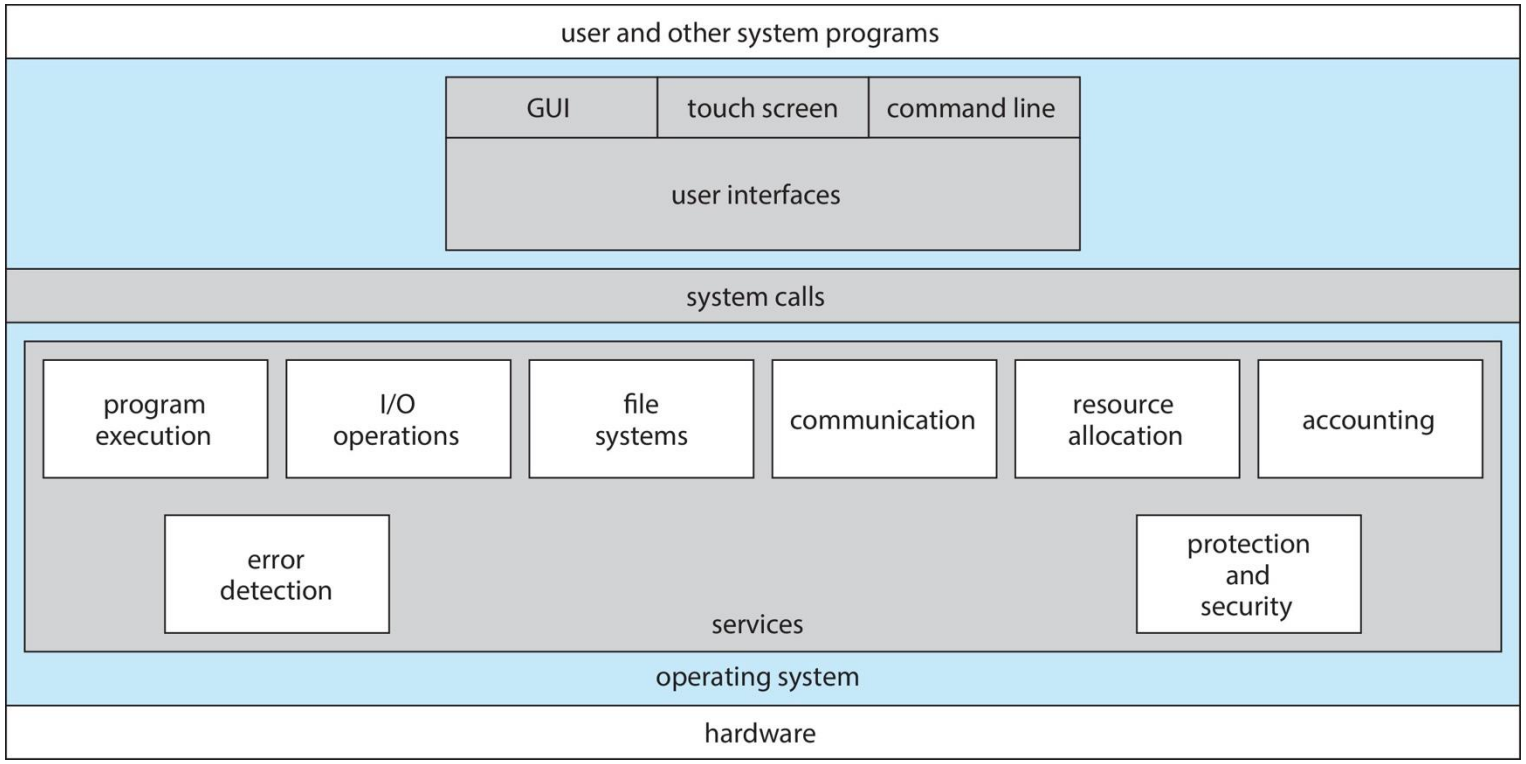
  3- Ability to evolve.

3- Easy of evolution of an OS.

- Hardware upgrades plus new types of hardware.
- New services.
- Fixes. Any OS has faults that are discovered over the course of time.



Operating system (OS) objectives and functions



Operating system (OS) objectives and functions

OS services

- Operating systems provide an environment for execution of programs and services to programs and users.
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line** (**CLI**), **Graphics User Interface** (**GUI**), **touch-screen, Batch.**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error).
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device.
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.

Operating system (OS) objectives and functions

OS services

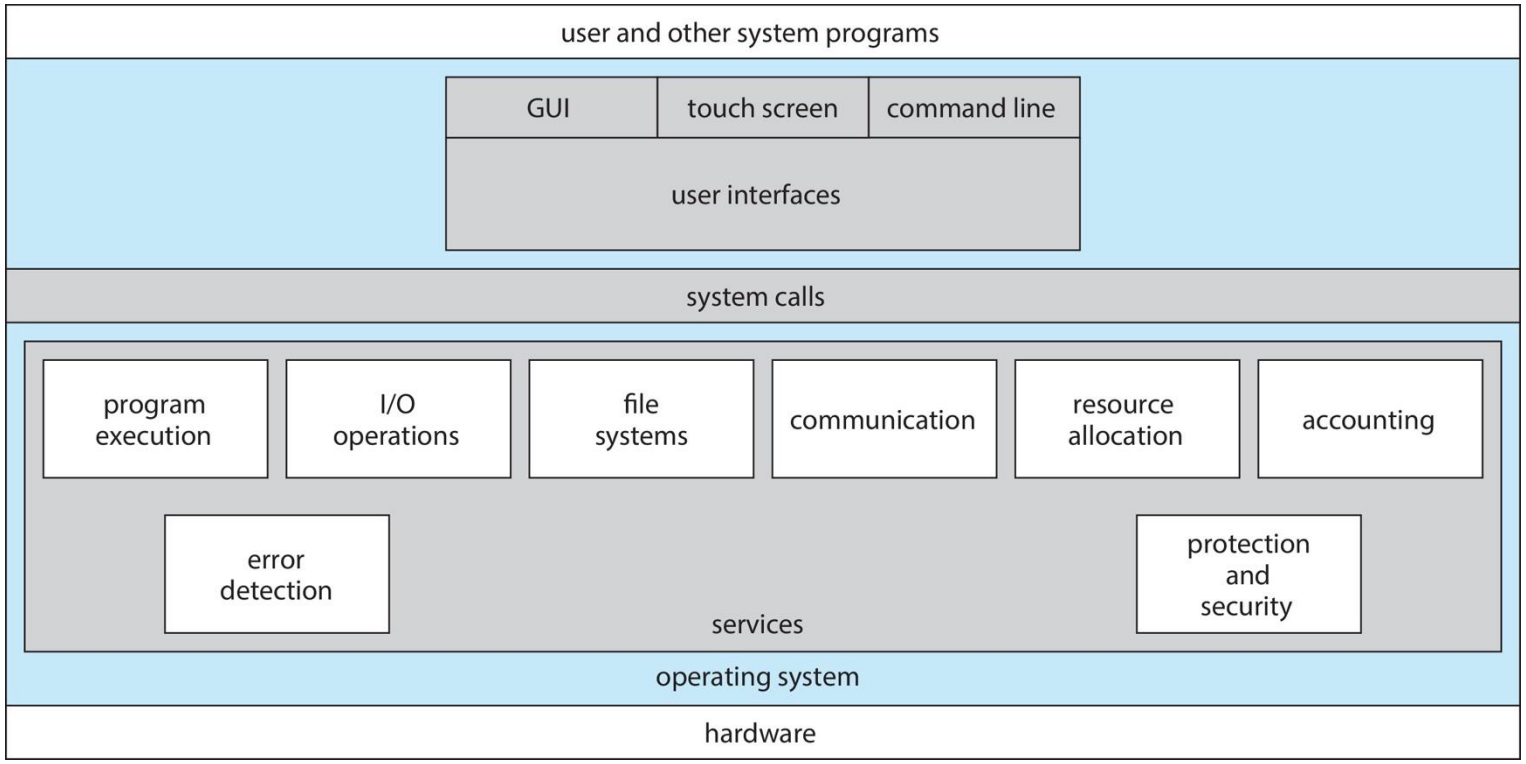
- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network.
    - Communications may be via shared memory or through message passing (packets moved by the OS).
  - **Error detection** – OS needs to be constantly aware of possible errors.
    - May occur in the CPU and memory hardware, in I/O devices, in user program.
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing.
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system.

Operating system (OS) objectives and functions

OS services

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing.
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them.
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Logging -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other.
    - **Protection** involves ensuring that all access to system resources is controlled.
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts.

Operating system (OS) objectives and functions



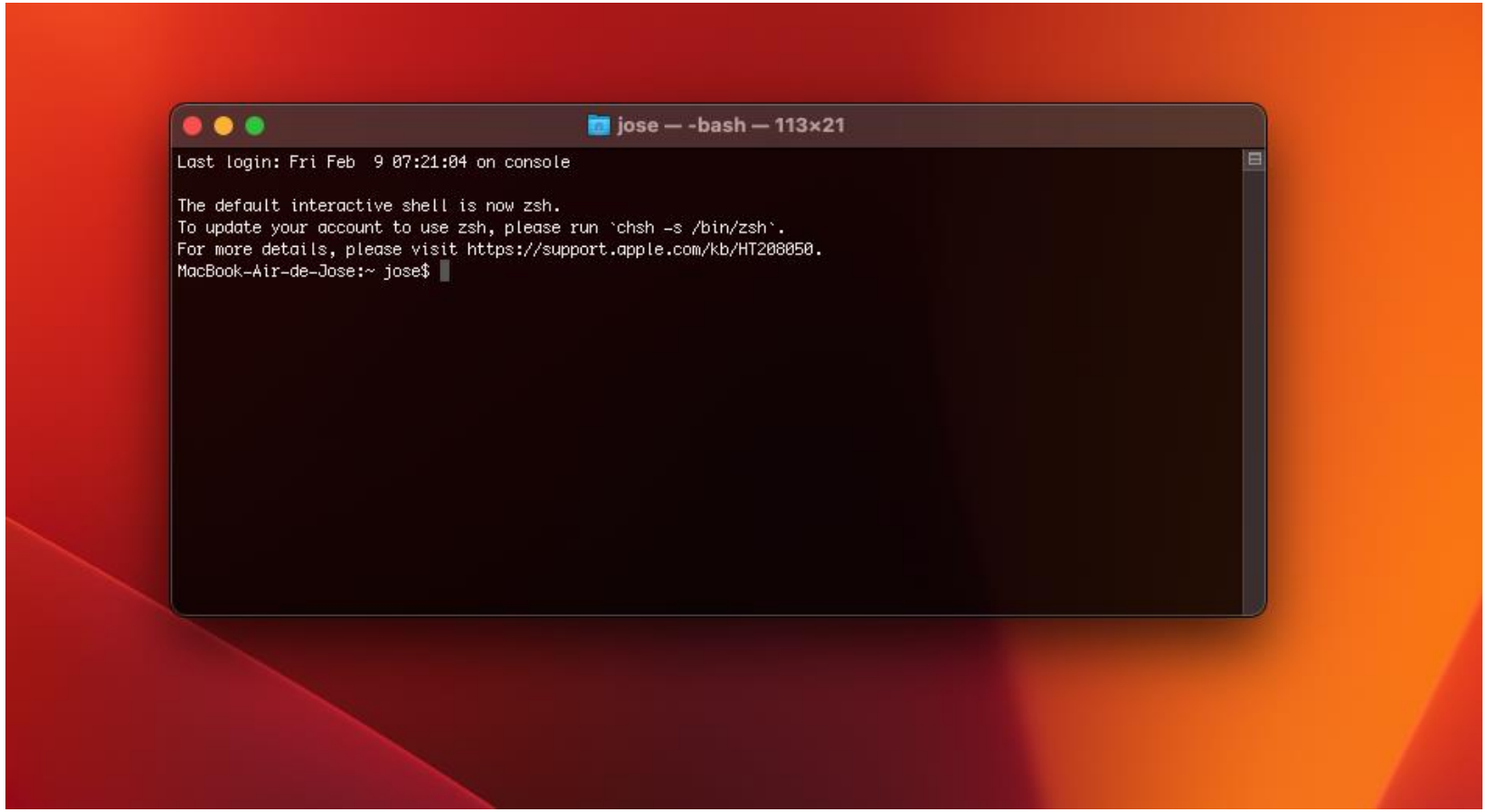
Operating system (OS) objectives and functions

Command Line Interpreter

- CLI allows direct command entry.
- Sometimes implemented in kernel, sometimes by systems program.
- Sometimes multiple flavors implemented – **shells.**
- Primarily fetches a command from user and executes it.
- Sometimes commands built-in, sometimes just names of programs.
  - If the latter, adding new features doesn't require shell modification.

Operating system (OS) objectives and functions

Command Line Interpreter



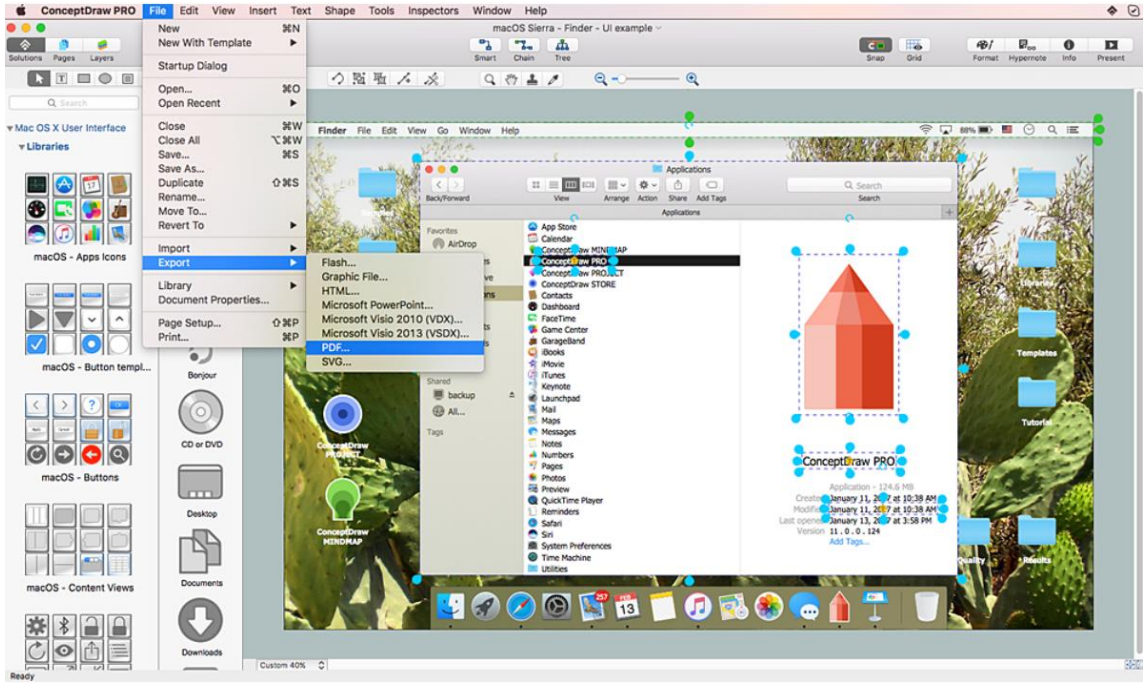
Operating system (OS) objectives and functions

User operating system graphical interface (GUI)

- User-friendly desktop metaphor interface.
  - Usually mouse, keyboard, and monitor.
  - Icons represent files, programs, actions, etc.
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a folder).
  - Invented at Xerox PARC.
- Many systems now include both CLI and GUI interfaces.
  - Microsoft Windows is GUI with CLI "command" shell.
  - Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available.
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME).

Operating system (OS) objectives and functions

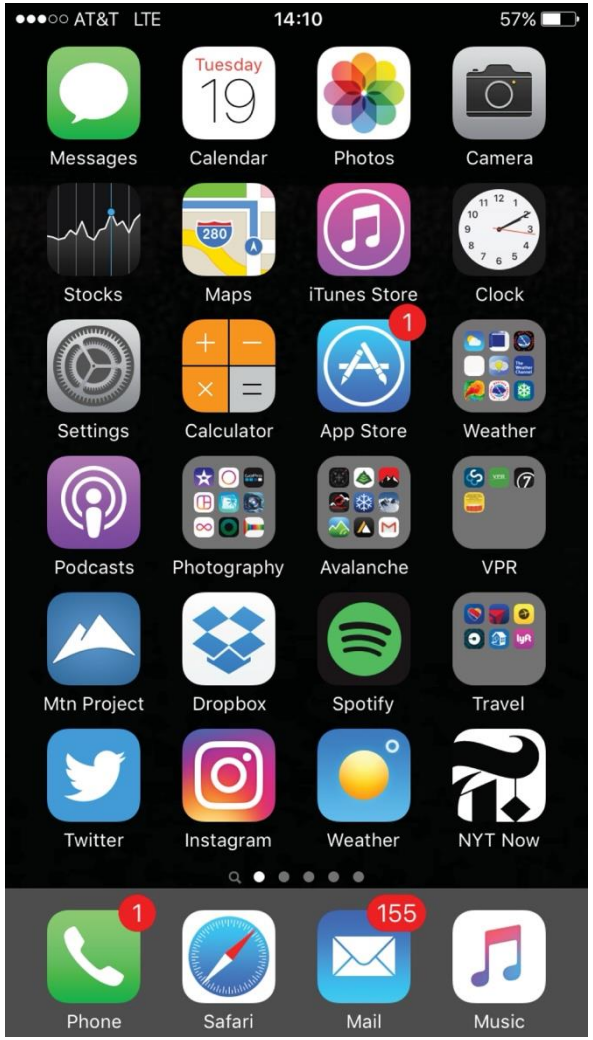
MAC OS X GUI



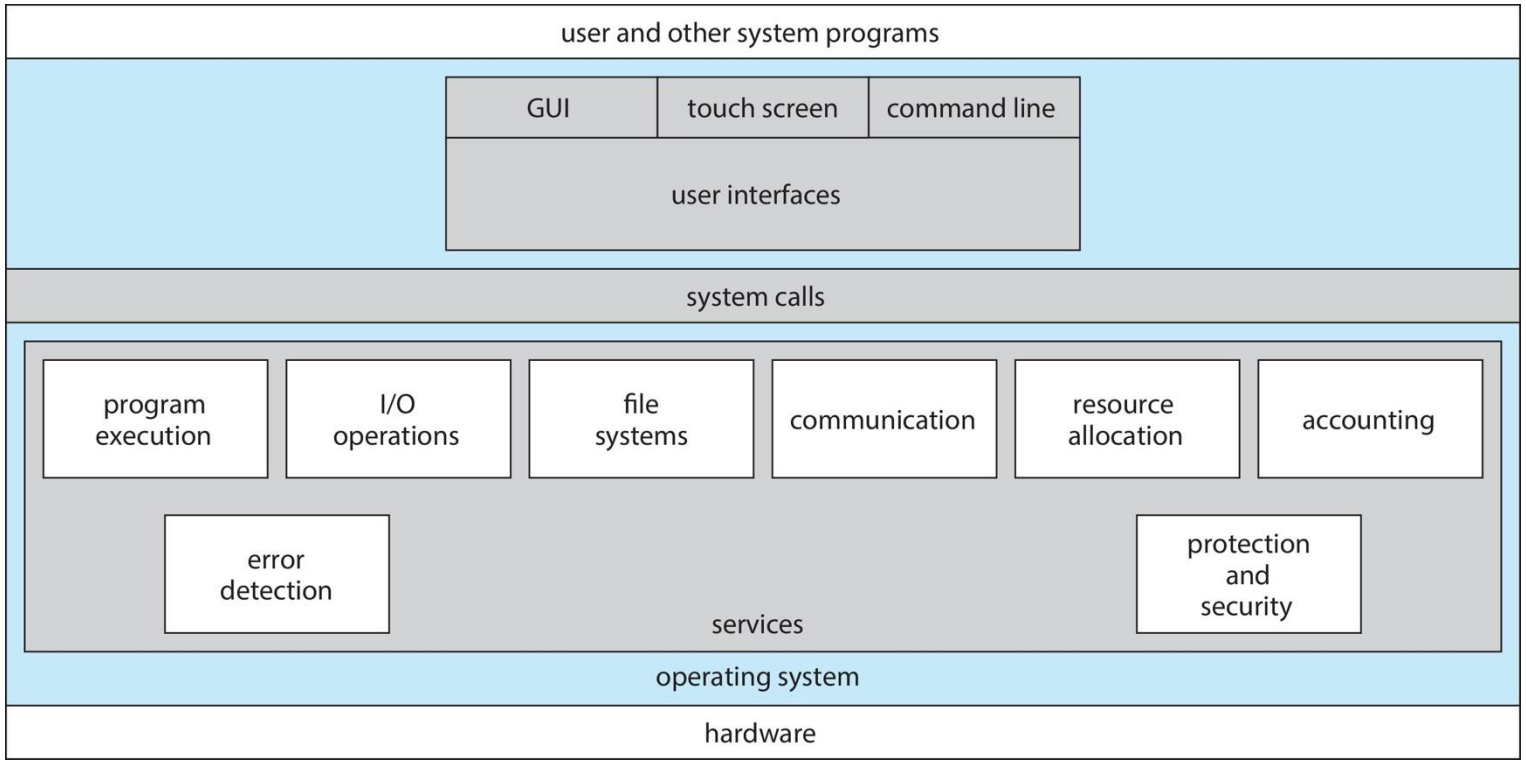
Operating system (OS) objectives and functions

Touchscreen user interface

- Touchscreen devices require new interfaces.
  - Mouse not possible or not desired.
  - Actions and selection based on gestures.
  - Virtual keyboard for text entry.
- Voice commands.



Operating system (OS) objectives and functions



Operating system (OS) objectives and functions

System Calls

- Programming interface to the services provided by the OS.
- Typically written in a high-level language (C or C++).
- Mostly accessed by programs via a high-level **Application Programming Interface** (**API**) rather than direct system call use.
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM).

Operating system (OS) objectives and functions

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

Operating system (OS) objectives and functions

Types of System Calls

- Process control
  - create process, terminate process.
  - end, abort.
  - load, execute.
  - get process attributes, set process attributes.
  - wait for time.
  - wait event, signal event.
  - allocate and free memory.
  - Dump memory if error.
  - Debugger for determining bugs, single step execution.
  - Locks for managing access to shared data between processes.

Operating system (OS) objectives and functions

Types of System Calls

- File management
  - create file, delete file.
  - open, close file.
  - read, write, reposition.
  - get and set file attributes.
- Device management
  - request device, release device.
  - read, write, reposition.
  - get device attributes, set device attributes.
  - logically attach or detach devices.

Operating system (OS) objectives and functions

Types of System Calls

- Information maintenance
  - get time or date, set time or date.
  - get system data, set system data.
  - get and set process, file, or device attributes.
- Communications
  - create, delete communication connection.
  - send, receive messages if message passing model to host name or process name.
    - From client to server.
  - Shared-memory model create and gain access to memory regions.
  - transfer status information.
  - attach and detach remote devices.
- Protection
  - Control access to resources.
  - Get and set permissions.
  - Allow and deny user access.

Operating system (OS) objectives and functions

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

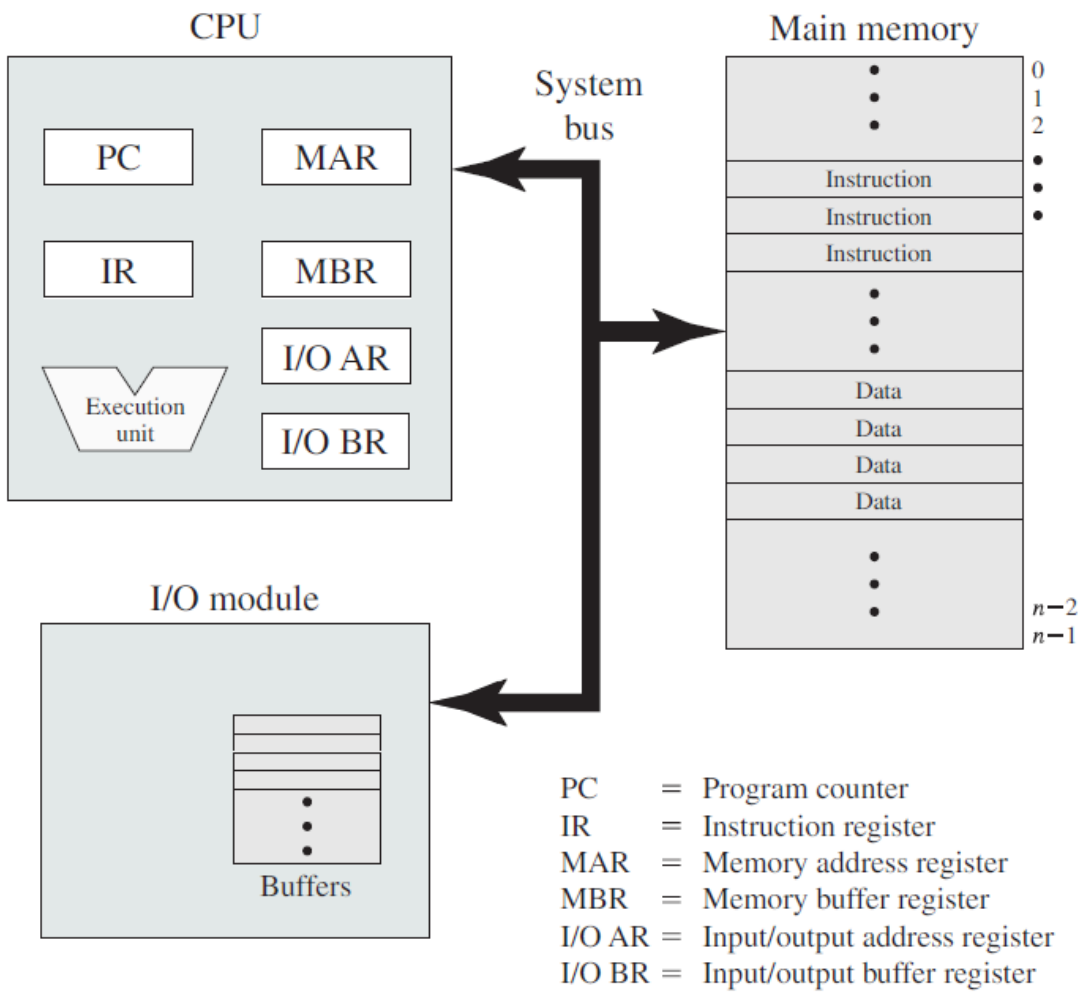
	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitlializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`

Contents

1) Objectives and functions
2) The evolution of operating systems
3) Major achievements
   - Processes
   - Memory management
   - Information protection and security
   - Scheduling and resource management
4) Features of modern operating systems
   - Microkernel architecture
   - Multithreading
   - Symmetric multiprocessing
   - Distributed operating systems
5) UNIX systems

The evolution of operating systems (OS)

The **OS** have evolved throughout the time with the **hardware.** They maintain a close relationship.



Current computer architecture and its basic hardware components:

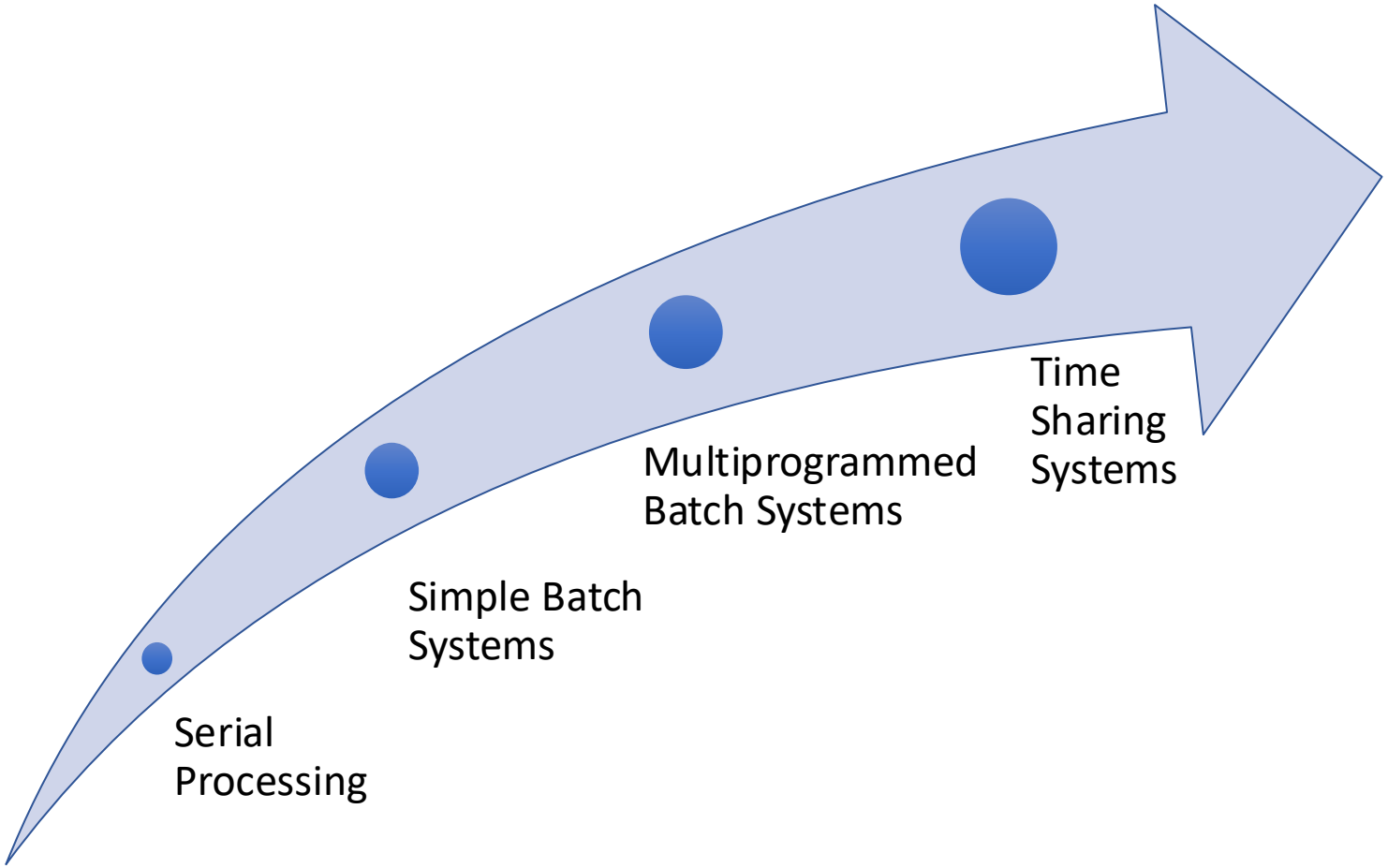
Processor. It is referred as CPU when there is just one.

Main memory.

I/O modules.

System bus.

The evolution of operating systems (OS)



The evolution of operating systems (OS)

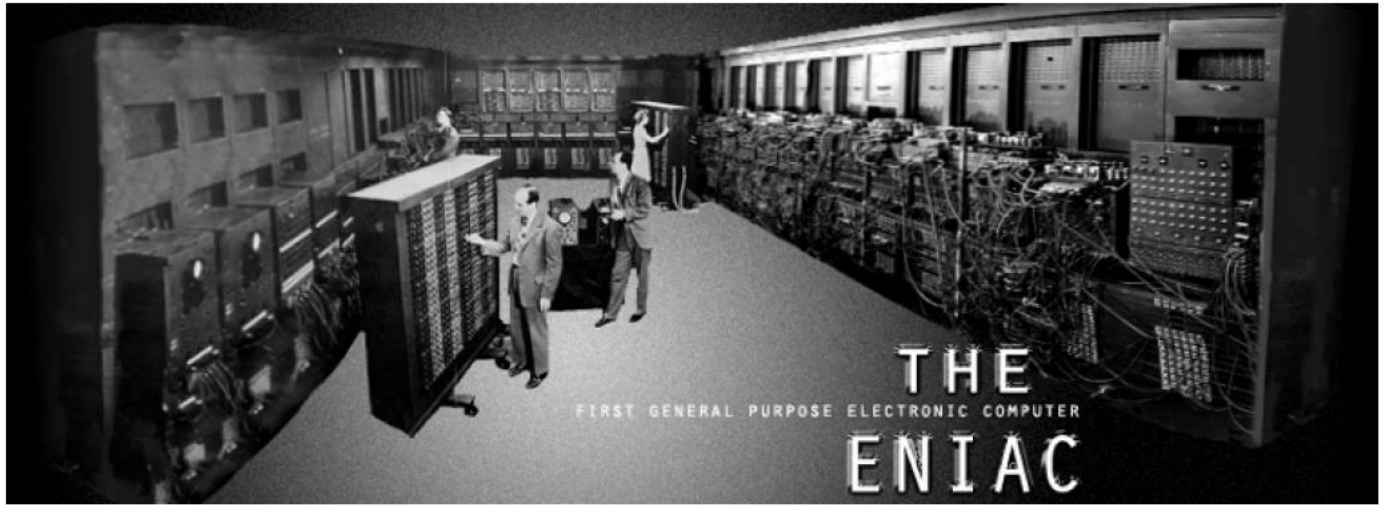
1- Serial processing

1940s-1950s: the programmer interacted directly with the computer hardware. There was no OS.

Users have access to the computer in series. It is not very efficient.

The programming was done physically, modifying the wiring each time you wanted to perform a different operation.

Instead of having to modify the connections of the machine, punched cards appeared, which allowed to write programs on a card.



Computers were very expensive. It was important to maximize processor utilization.

The evolution of operating systems (OS)

2- Simple batch systems

The concept of a batch OS was developed by General Motors for use on an IBM 701 in the **mid-1950s.**

Central idea: use a piece of software known as the **monitor (batch OS)**. It relies on the **ability** of the **processor** to **fetch instructions** from various portions of main memory to **alternately seize** and **relinquish control.**

The user no longer has direct access to the processor.

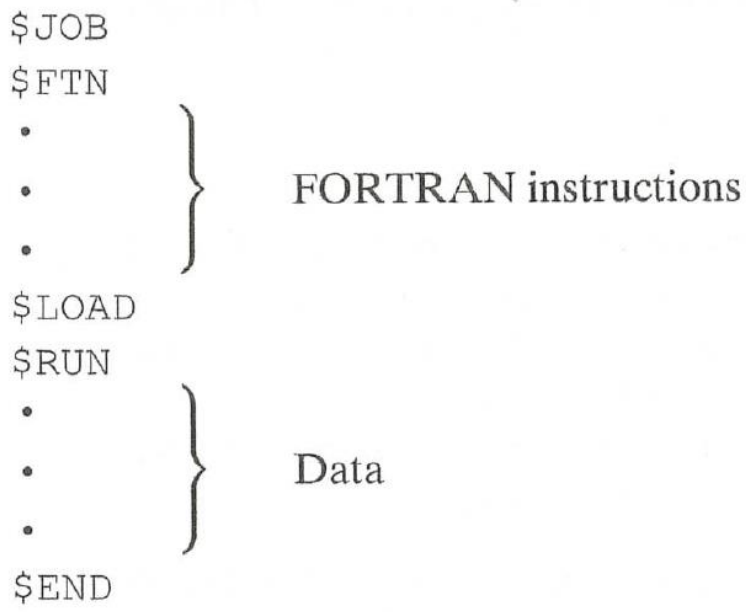
The first programming language appears: **FORTRAN.**

JCL (Job Control Language). This is a special programming language to provide instructions to the monitor. Monitor reads instructions in this language.

FORTRAN instructions and data are on a separate punched card or tape.

The user submits the job on cards or tape to a computer operator.

This **machine** has **already** a **basic idea** of **OS**: control start and end of jobs, read data, write of results…



The evolution of operating systems (OS)

2- Simple batch systems

Desirable hardware features:

- **Memory protection:** while the user program is executing, it must not alter the memory area containing the monitor.
- **Timer:** used to prevent a single job from monopolizing the system.
- **Privileged instructions:** certain instructions can be executed only by the monitor.
- **Interrupts:** Gives the OS more flexibility in relinquishing or regaining control from user programs.

Concept of modes of operation:

- **User mode:** certain areas of memory are protected from the user´s use.
- **Kernel mode:** privileged instructions may be executed. Protected areas of memory may be accessed.

The evolution of operating systems (OS)

3- Multiprogrammed batch systems

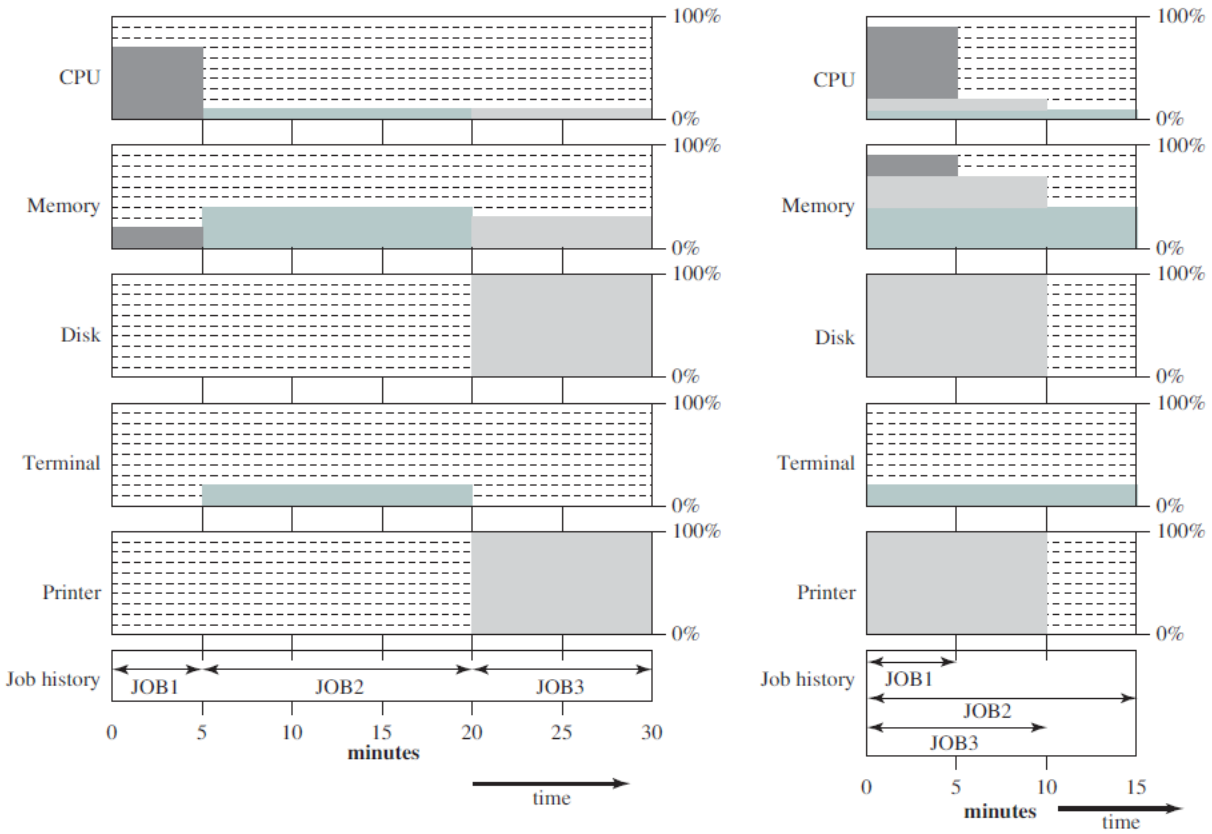
Even with an automatic job by a simple batch OS, the processor is often idle.

Problem: I/O devices are slow compared to the processor.

Multiprogramming or multitasking: more than one user program. When one job needs to wait for I/O, the processor can switch to the other job, which is not waiting for I/O.

To have several jobs ready to run, they must be kept in main memory. Requirement of **memory management.**

The processor must decide which job to run. This decision needs an algorithm **for scheduling.**



The evolution of operating systems (OS)

4- Time-sharing systems (1960s)

- Desirable to provide a mode in which the user interacts with the computer (instructions received through the terminal).
- Computers were big and costly. That´s the reason to share it.
- **Time sharing:** processor time is shared among multiple users.
- **Multiple users** simultaneously access the system through terminals.
- One of the first time-sharing OS was the **CTSS** (Compatible Time-Sharing System).
- **CTSS** approach is primitive compared to present-day time sharing.
- A system clock generated **interrupts** at a specific rate. At each clock interrupt, the OS regained control and could assign the processor to another user (time slicing).
- Jobs always loaded in the same portion of memory. No need for relocation techniques.
- **New problems:** If multiple jobs are in memory, then they must be protected from interfering with each other, for example, modifying each other data.

The evolution of operating systems (OS)

4- Time-sharing systems (1960s)

https://www.youtube.com/watch?v=Q07PhW5sCEk

The evolution of operating systems (OS)

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

Contents

1) Objectives and functions
2) The evolution of operating systems
3) Major achievements
   - Processes
   - Memory management
   - Information protection and security
   - Scheduling and resource management
4) Features of modern operating systems
   - Microkernel architecture
   - Multithreading
   - Symmetric multiprocessing
   - Distributed operating systems
5) UNIX systems

Major achievements

- OS are among the most complex pieces of software ever developed.
- 4 main theoretical advances in the development of OS:
  - Processes.
  - Memory management.
  - Information protection and security.
  - Scheduling and resource management.

Major achievements: processes

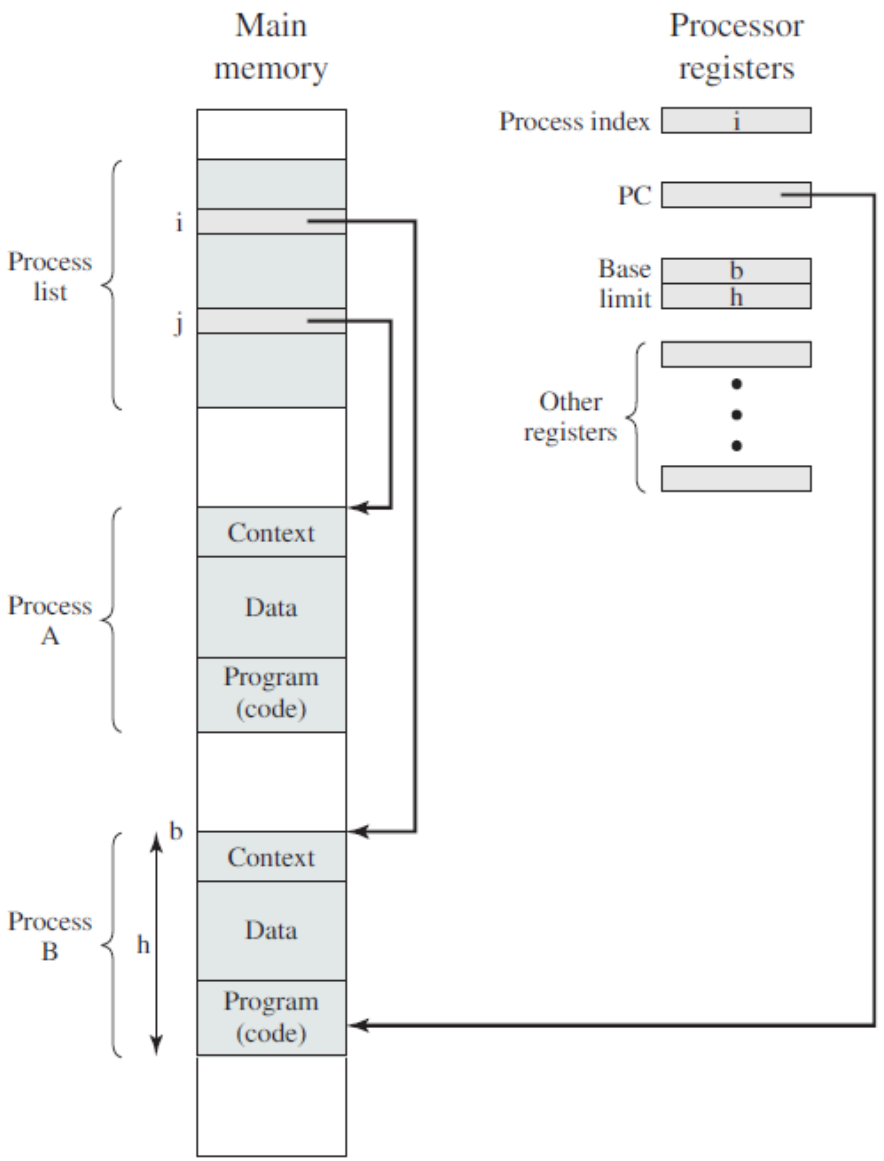
- The term **process** was first used by the designers of **Multics** in the 1960s.
- **Process** is a more general concept than **job**.
- Definitions:
  - A program in execution.
  - An instance of a program running on a computer.
  - The entity that can be assigned to and executed on a processor.
  - A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources.

Major achievements: processes

- The concept of the process provides a **systemic** way to monitor and control the various programs executing on the processor.
- A process contains:
  - An **executable program.**
  - The associated **data** needed by the program.
  - The **execution context** of the program (**process state**). Internal data uses by the OS to supervise and control the process. This **structure** of data **allows** the development of techniques for ensuring **coordination** and **cooperation** among **processes**.

    - Information of processor registers (Program counter, data registers).

    - Priority of the process,..

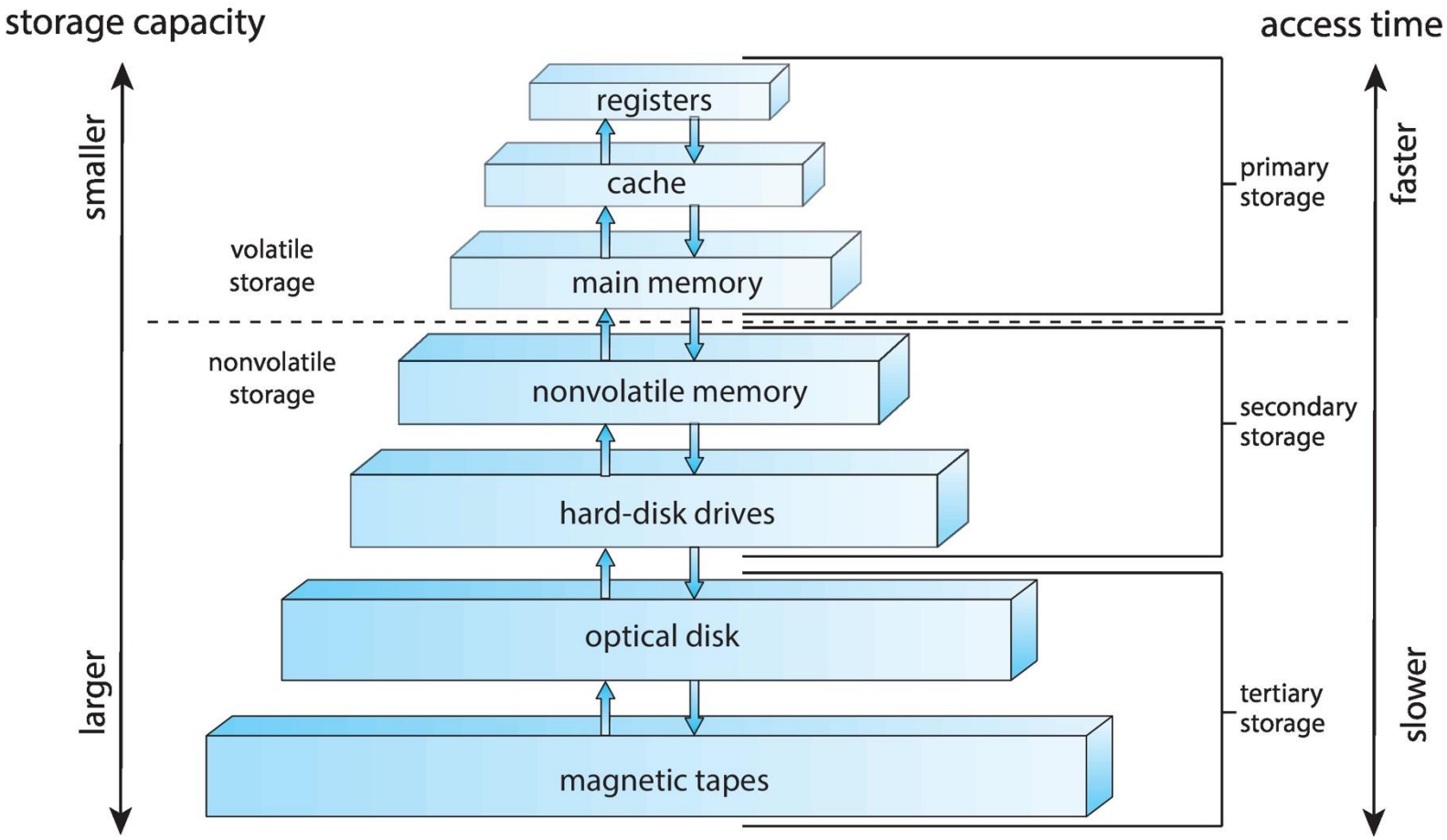


Major achievements: memory management

- Principal storage management responsibilities of OS:
  - Process isolation.
  - Automatic allocation and management.
  - Support of modular programming.
  - Protection and access control.
  - Long-term storage.

- **OS** meet these requirements with **virtual memory** and **file system** facilities.
  - **File system** implements a long-term store.
  - **Virtual memory** is a facility that allows programs to address memory from a logical point of view, without considering the amount of the main memory physically available.
  - **Paging systems:** allow processes to be comprised of a number of fixed-size block, called pages. Each page of a process may be located anywhere in main memory. It provides for a dynamic mapping between the virtual address used in the program and a real address (physical address in main memory).

Major achievements: memory management



Major achievements: information protection and security

Much of the work in security and protection grouped in (more important in time-sharing systems and computer networks):

1- **Availability**. Protection of the system against interruption.

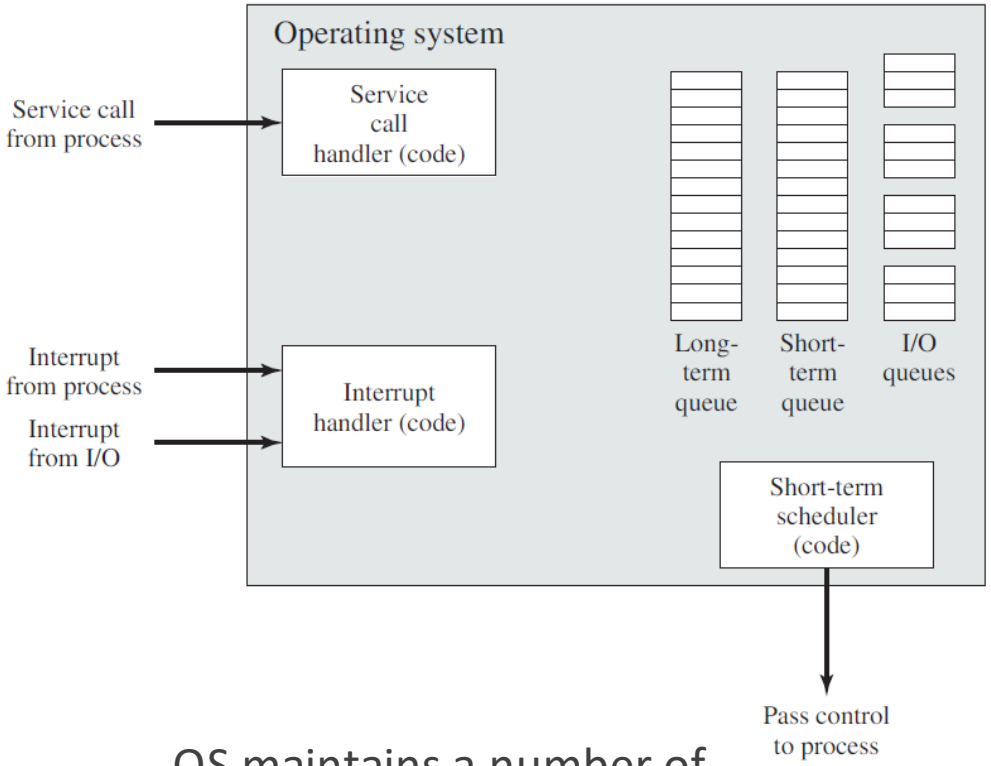
2- **Confidentiality**. Users cannot read unauthorized data.

3- **Data integrity**. Protection of data from unauthorized modification.

4- **Authenticity**. Proper verification of the identity of users and the validity of messages or data.

Major achievements: scheduling and resource management

- Main responsibility of the **OS** is to **manage** the various **resources** available (main memory, I/O devices, processors) and to **schedule** their **use by** the various active **processes**.
- **Fairness: Equal** and **fair access** to the resources for processes.
- **Differential responsiveness:** OS need to discriminate among different classes of jobs. The OS should attempt to make **allocation** and **scheduling decisions** to meet the **total** set of **requirements**.
- **Efficiency:** OS should attempt to maximize **throughput**, **minimize response time** and **accommodate** as many **users** as possible in case of time sharing.



OS maintains a number of **queues** (list of processes waiting for some resource).

Contents

1) Objectives and functions
2) The evolution of operating systems
3) Major achievements
   - Processes
   - Memory management
   - Information protection and security
   - Scheduling and resource management
4) Features of modern operating systems
   - Microkernel architecture
   - Multithreading
   - Symmetric multiprocessing
   - Distributed operating systems
5) UNIX systems

Modern operating systems

Microkernel architecture

- Few essential features to the kernel. Other OS services are provided by processes (running in user mode).
- Simplification of the implementation.
- It is well suited for distributed environments.

Modern operating systems

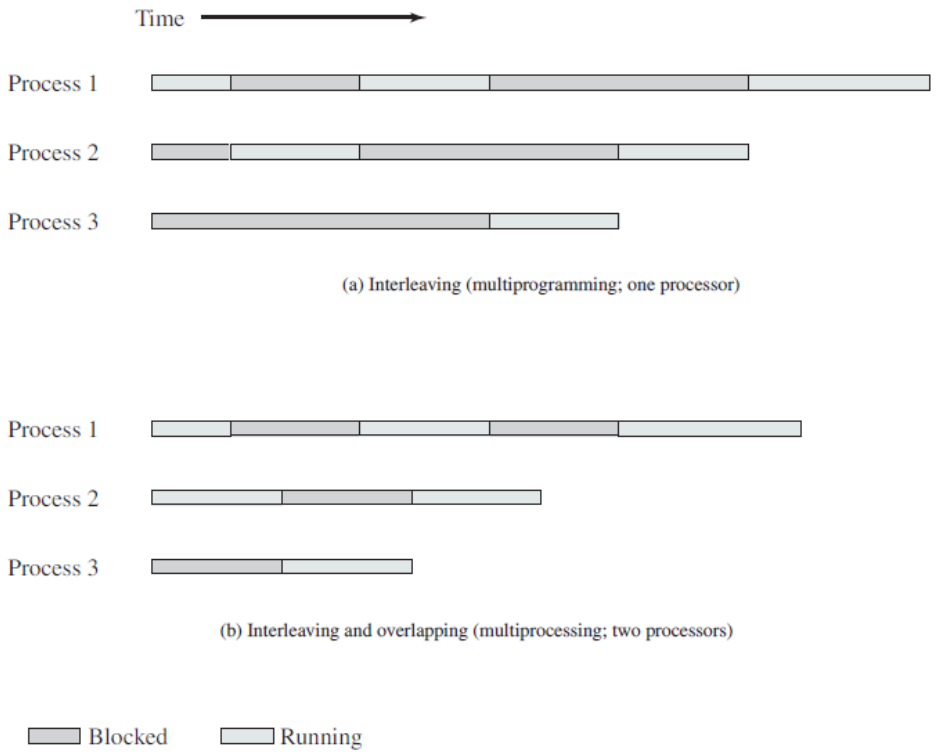
Multithreading

- It is a technique in which a process, executing an application, is divided into threads that can run concurrently.
- **Thread:** a dispatchable unit of work. A thread executes sequentially and is interruptible so the processor can turn to another thread.
- **Process:** a **collection** of **one or more threads** and associated system resources. It is close to a program in execution. By breaking a single application into multiple threads, the programmer has control over the modularity of the application and the timing of events.
- Example: a database server that listens for numerous client requests.

Modern operating systems

Symmetric multiprocessing (SMP)

- Two types:
  a. **Asymmetric multiprocessing** – each processor is assigned a special task.
  b. **Symmetric multiprocessing** – each processor performs all tasks.
- It is a term that refers to a computer hardware architecture.
- The OS of an SMP schedules processes or threads across all of the processors.
- Advantages:
  a. Performance.
  b. Availability.
  c. Incremental growth.
  d. Scaling.
- The OS must provide tools and functions to exploit the parallelism in an SMP system.



Multiprogramming and multiprocessing.

Modern operating systems

Distributed operating systems

- Distributed system: Collection of separate, possibly heterogeneous, systems networked together.
- **Network** is a communications path, **TCP/IP** most common.
- **Network Operating System** provides features between systems across network.
- Communication scheme allows systems to exchange messages.
- Illusion of a single system.

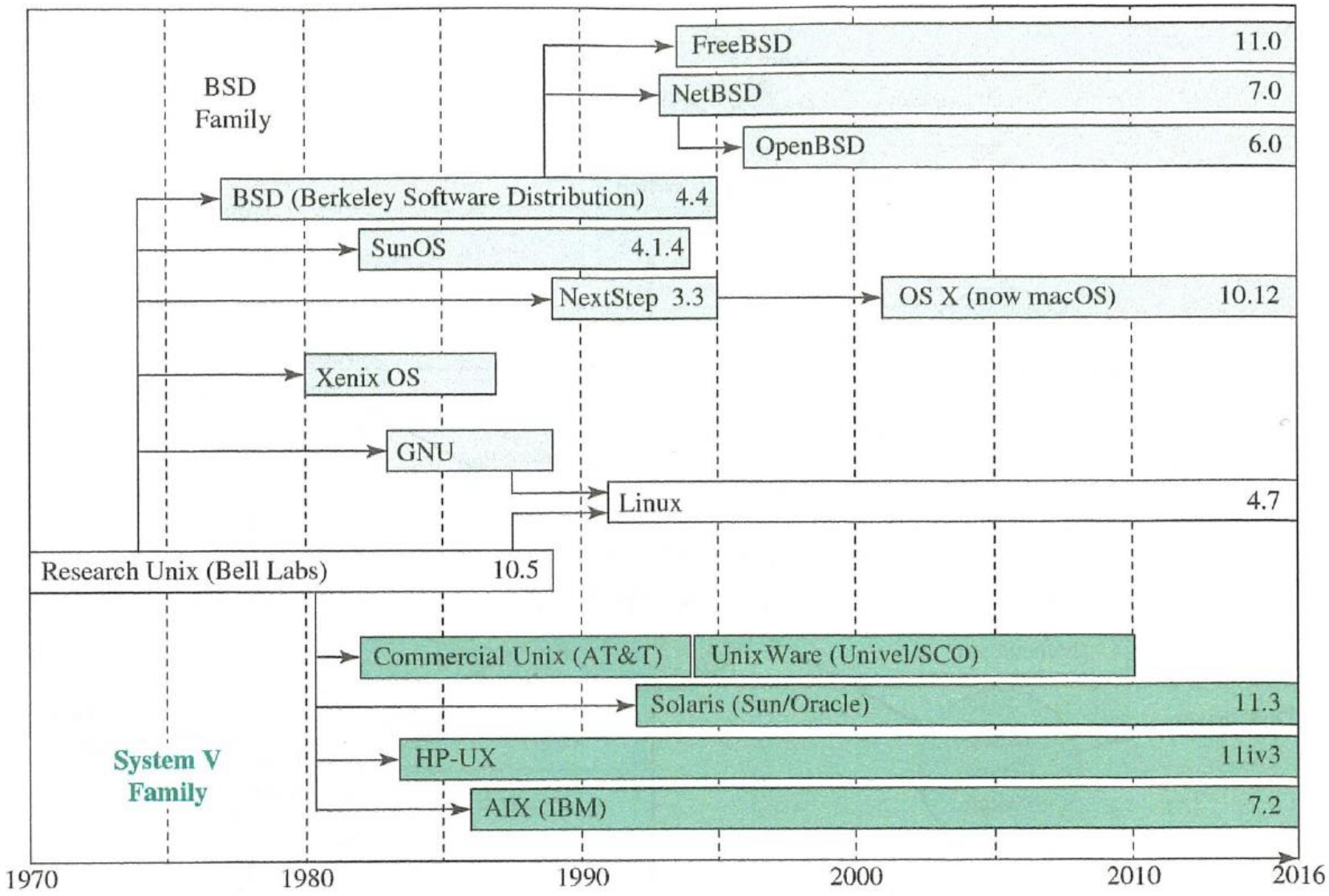
Contents

1) Objectives and functions
2) The evolution of operating systems
3) Major achievements
   - Processes
   - Memory management
   - Information protection and security
   - Scheduling and resource management
4) Features of modern operating systems
   - Microkernel architecture
   - Multithreading
   - Symmetric multiprocessing
   - Distributed operating systems
5) UNIX systems

UNIX systems

- **UNIX** was initially developed at **Bell Labs** and became operational in 1970.
- Milestones:
  - PDP-7 to PDP11 (Unix would be an OS for all computers).
  - Rewriting of UNIX in the programming language C.
- In 1974, the UNIX system was described in a technical journal for the first time.

UNIX family tree

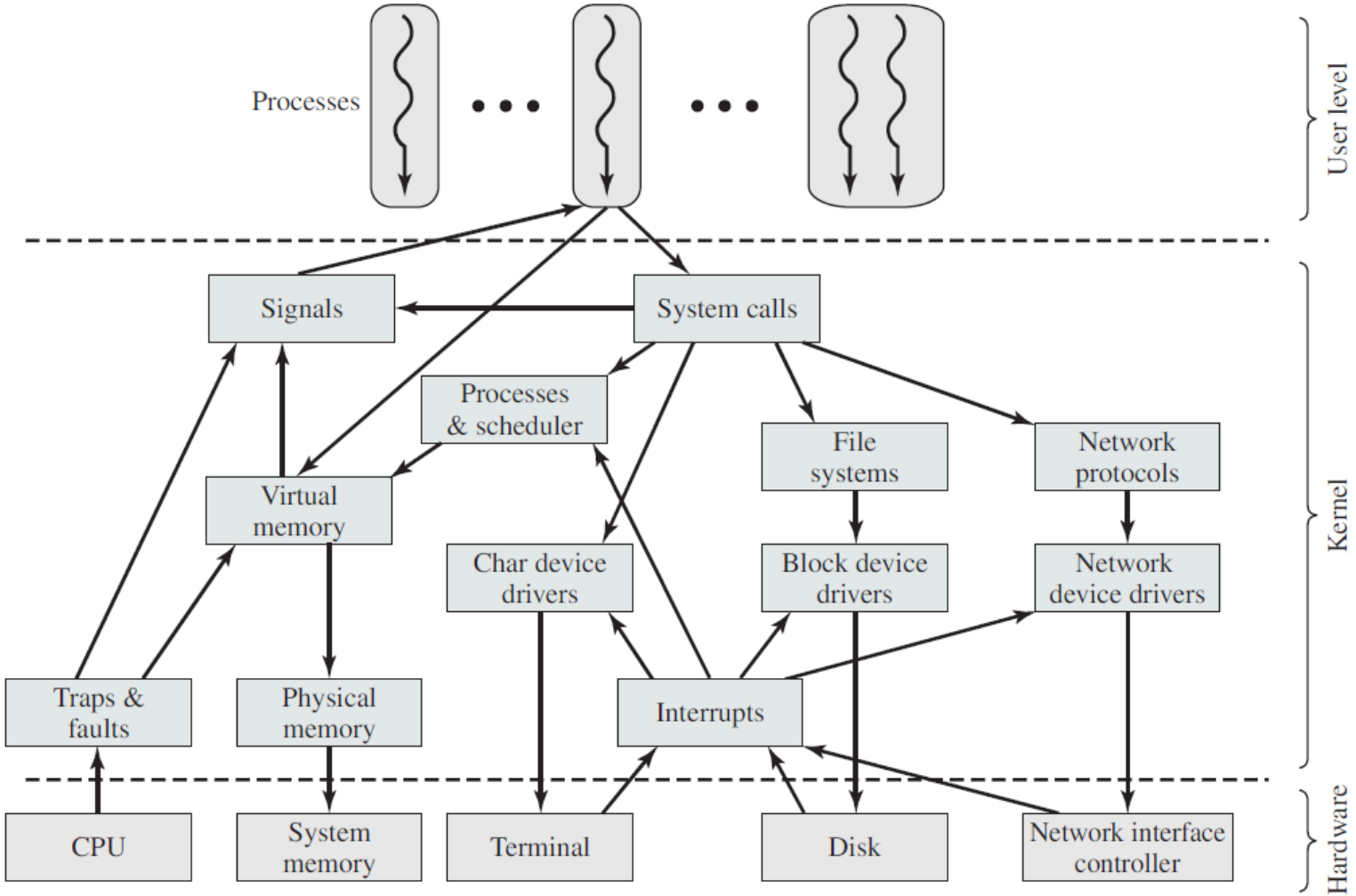


Linux

- The initial version of **Linux** was written by **Linus Torvalds**, a Finnish student of computer science (1991).
- Since then, a group of people, collaborating over the Internet, have contributed to the development, under the control of Torvalds.
- Linux is **free** and the **source code** is available. Visit: http://www.kernel.org
- Linux has now made significant penetration into the corporate world, because of the quality of the kernel.
- Most UNIX kernels are **monolithic** (one that includes virtually all the functionality in one large block of code that runs as a single process with a single address space).
- Linux is structured as a collection of modules, a number of which can be automatically loaded and unloaded on demand (file system, device driver), although is considered monolithic.
- A module does not execute as its own process or thread, although it can create kernel threads for various purposes. A module is executed in kernel mode on behalf of the current process.

Linux: Kernel components

- **Signals:** the kernel uses signals to call into a process.
- **System calls:** a process requests a specific kernel service.
- **Processes and scheduler:** create, manage, and schedules processes.
- **Virtual memory:** allocates and manages virtual memory for processes.
- **File systems**
- **Network protocols:** sockets interface to users for the TCP/IP protocol suite.
- **Network device drivers:** manage network interface cards and communications ports that connect to network devices, such as bridges and routers.



Linux: Kernel components

Some Signals

SIGHUP	Terminal hangup	SIGCONT	Continue
SIGQUIT	Keyboard quit	SIGTSTP	Keyboard stop
SIGTRAP	Trace trap	SIGTTOU	Terminal write
SIGBUS	Bus error	SIGXCPU	CPU limit exceeded
SIGKILL	Kill signal	SIGVTALRM	Virtual alarm clock
SIGSEGV	Segmentation violation	SIGWINCH	Window size unchanged
SIGPIPT	Broken pipe	SIGPWR	Power failure
SIGTERM	Termination	SIGRTMIN	First real-time signal
SIGCHLD	Child status unchanged	SIGRTMAX	Last real-time signal

Some System Calls

File system Related	
close	Close a file descriptor.
link	Make a new name for a file.
open	Open and possibly create a file or device.
read	Read from file descriptor.
write	Write to file descriptor.
Process Related	
execve	Execute program.
exit	Terminate the calling process.
getpid	Get process identification.
setuid	Set user identity of the current process.
prtrace	Provides a means by which a parent process may observe and control the execution of another process, and examine and change its core image and registers.

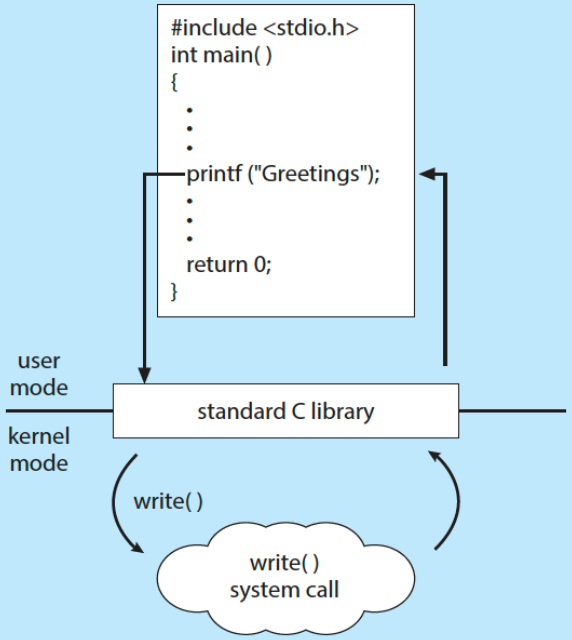
Socket (networking) Related	
bind	Assigns the local IP address and port for a socket. Returns 0 for success and −1 for error.
connect	Establish a connection between the given socket and the remote socket associated with sockaddr.
gethostname	Return local host name.
send	Send the bytes contained in buffer pointed to by *msg over the given socket.
setsockopt	Set the options on a socket
Miscellaneous	
create_module	Attempt to create a loadable module entry and reserve the kernel memory that will be needed to hold the module.
fsync	Copy all in-core parts of a file to disk, and waits until the device reports that all parts are on stable storage.
query_module	Request information related to loadable modules from the kernel.
time	Return the time in seconds since January 1, 1970.
vhangup	Simulate a hangup on the current terminal. This call arranges for other users to have a “clean” tty at login time.

Standard C library example

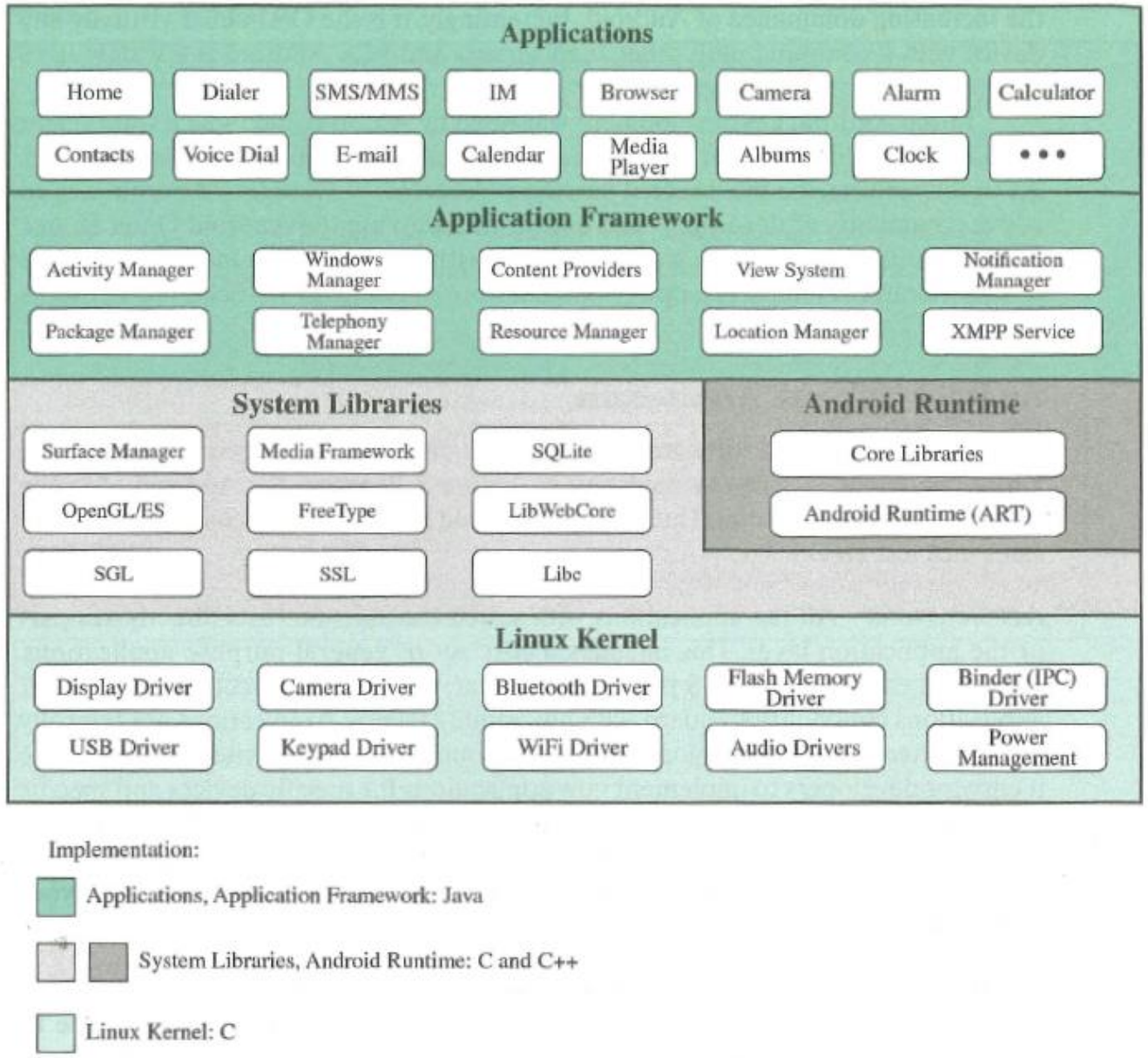
C program invoking `printf()` library call, which calls `write()` system call

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



Android OS overview



Universidad
LOYOLA

Universidad
LOYOLA